Building Resilient PHP Apps

with an Event-Driven Mindset

by **Savio Resende**

PHPTek 2026

A Friday Afternoon…

> Your team ships an hotfix. By Monday, 10,000 order statuses silently flipped from *processing* to *shipped* — but nothing actually shipped.

What was the correct state? You don't know. The history is gone.

The Real Problem

- Traditional systems **overwrite state** on every update
- You know **what is**. Not **what was**. Never **why**.
- History tables capture snapshots — not intent

Talk Objective

1. **What** is event sourcing — and why now?
2. **How** to apply it in PHP (Laravel + Spatie)
3. **Where** it pays off — and where it doesn't

Same Feature, Two Mindsets

TRADITIONAL

- INSERT order row
- UPDATE status = 'paid'
- UPDATE status = 'shipped'
- Logs? Maybe.

EVENT-SOURCED

- OrderPlaced
- PaymentReceived
- OrderShipped
- Every fact, immutable

The Mindset Shift

Stop modeling state.
Start modeling behavior.

State becomes a consequence — not the source of truth.

Core Concepts

Events

Event Store

Aggregates

Projections

Reactors

Events

- Immutable facts about something that **happened**
- Always past tense — completed actions
- Never deleted, never modified
- Mistakes? Record a compensating event (Cancelled, Refunded)

OrderPlaced | PaymentReceived | OrderShipped

The Event Store

- Append-only log
- Add to the end — never modify, never remove
- Your system's single source of truth

Think: git, but for your domain.

Aggregates

- Where business rules live
- Cluster of related data that must stay consistent
- Loads history → applies rules → emits new events

> Can't ship an unpaid order. Can't cancel a delivered one.

Projections

- Read-optimized views, built from events
- One projection per question your app needs to answer
- New question? Build a new projection — replay all events

You can answer questions you didn't know to ask.

Reactors

- Side effects triggered by events
- Aggregates stay clean — they don't know about emails or alerts
- Reactors respond to facts

DangerousHeartRate → page the on-call physician.

The Mental Model

> state = fold(events, initialState)

Like a bank balance — just the sum of every deposit minus every withdrawal.

The Flow


Command
Aggregate
Event
Projector
Event also fans out to →
Reactor

Examining the Code

1. E-commerce Dashboard
2. Health Metrics Tracker

Heart rate, blood pressure, oxygen — events all the way down.

The Event

```php
use Spatie\EventSourcing\StoredEvents\ShouldBeStored;

class HeartRateRecorded extends ShouldBeStored
{
    public function __construct(
        public string $patientId,
        public int $bpm,
        public string $recordedAt,
    ) {}
}
```

The Aggregate

```php
use Spatie\EventSourcing\AggregateRoots\AggregateRoot;

class PatientAggregate extends AggregateRoot
{
    public function recordHeartRate(int $bpm): self
    {
        $this->recordThat(new HeartRateRecorded(
            patientId: $this->uuid(),
            bpm: $bpm,
            recordedAt: now()->toIso8601String(),
        ));

        return $this;
    }
}
```

The Projector

```php
use Spatie\EventSourcing\EventHandlers\Projectors\Projector;

class HealthDashboardProjector extends Projector
{
    public function onHeartRateRecorded(HeartRateRecorded $event): v
    {
        HealthReading::writeable()->create([
            'patient_id' => $event->patientId,
            'metric'     => 'heart_rate',
            'value'      => $event->bpm,
            'recorded_at'=> $event->recordedAt,
        ]);
    }
}
```

The Reactor

```php
use Spatie\EventSourcing\EventHandlers\Reactors\Reactor;

class DangerousHeartRateReactor extends Reactor
{
    public function onHeartRateRecorded(HeartRateRecorded $event): v
    {
        if ($event->bpm > 150) {
            Alert::page(
                team: 'medical',
                patient: $event->patientId,
                bpm: $event->bpm,
            );
        }
    }
}
```

The Time Machine

```php
// Replay every event up to a moment in history
PatientAggregate::retrieve($patientId)
    ->replay(until: '2026-01-15 09:00');
```

Reconstruct exact state at any point in the past.

The Hard Parts

…because nothing this good is free.

Idempotency

- Same event delivered twice → projection double-counts
- Track which event IDs you've already processed
- Check before applying

A duplicate `PaymentReceived` is not a free coffee.

Event Versioning

- Six months in: `OrderPlaced` needs a new field
- Millions of old events don't have it
- Solution: upcasting — transform on load
- Or version events explicitly and handle each shape

Projection Rebuilds

50M

events. Now what?

- Snapshots — checkpoints to skip ahead

Testing

```php
// Given / When / Then
PatientAggregate::fake($patientId)
    ->given([
        new HeartRateRecorded($patientId, 80,  '...'),
        new HeartRateRecorded($patientId, 95,  '...'),
    ])
    ->when(fn ($p) => $p->recordHeartRate(160))
    ->assertRecorded([
        new HeartRateRecorded($patientId, 160, '...'),
    ]);
```

Declarative. Explicit. Surprisingly readable.

Adopting Incrementally

- **Strangler fig** — grow around the old system
- Pick the module with the most audit pain

When NOT to Use It

- A simple blog
- A CRUD admin panel
- Anywhere "what happened" doesn't matter

Event sourcing is a tool, not a religion.

Where It Earns Its Keep

Financial systems

Healthcare

E-commerce

Compliance-heavy

Anywhere history is a feature, not an afterthought.

The Real Shift

> What happened matters more than what is.

Capture facts. Derive state. Build systems that remember.

Resources

- github.com/lotharthesavior/es-example-2
- spatie.be/docs/laravel-event-sourcing
- Greg Young — *CQRS & Event Sourcing* (talks)
- Martin Fowler — *Event Sourcing* pattern

Thank You

savioresende.com

@lotharthesavior

https://joind.in/event/php-tek-2026/building-resilient-php-applications-with-an-event-driven-mindset